

KNOW IT

Happy Bank – Advanced

Máte za sebou základní
workshop?

Co byste si dnes chtěli
vyzkoušet?

Disclaimer

Na tomto workshopu se **nedozvíte** konkrétní postupy přesně pro vaši roli

Tento workshop slouží pro pochopení principů

Plán na dnešek

- Hook
 - Zapojení vlastní kontroly činností AI
- MCP
 - Tvorba vlastního serveru
- Agenti
 - Vlastní testovací pipeline
- Skills
 - Tvorba a validace vlastního skillu

Hook

- Vytvořte hook pro logování akcí AI v chatu
 - Začátek chatu
 - Konec odpovědi
 - Před a po spuštění toolu
 - Spuštění a ukončení sub-agenta
- Jak?
 - Začněte s `/create-hook` v chatu

Hook

- Vytvořte hook pro logování akcí AI v chatu
 - Začátek chatu
 - Konec odpovědi
 - Před a po spuštění toolu
 - Spuštění a ukončení sub-agenta
- Jak?
 - Začněte s `/create-hook` v chatu

Hook – bezpečnostní pojistka

- Může sloužit k blokování destruktivních příkazů
 - smazání souborů mimo git
 - přepsání historie gitu
 - mazání databází – SQL, S3, ...
- Například https://github.com/Dicklesworthstone/destructive_command_guard
 - modulární kontrola příkazů
 - integrace do různých harnesses
 - fail-open i fail-close strategii

Instalace prostředí

1. Nechte AI provést obsah souboru [INSTALL.md](#)
2. Nechte AI spustit uvicorn server podle [README.md](#)
3. Objevil se soubor [app.db](#)?
 - ANO → vypněte server
 - NE → zavolejte o pomoc

Tvorba vlastního MCP serveru

- Vytvořte MCP endpointy na vyčítání schématu databáze
 - získání jmen tabulek
 - získání sloupců dané tabulky
- Databáze je SQLite a nachází se v souboru `app.db`

MCP - zpomalení a dekompozice

- Časté problémy při vite codingu MCP serveru
 - Problém obsahuje mnoho částí. Není jasné, která zrovna nefunguje
- Zpomalení, zazoomování, navigování
 - Z jakých kroků se řešení skládá? → zeptejte se AI
 - Jak poznám u každého kroku, že mám hotovo? → zeptejte se AI
 - Problém s některým z kroků? → další dekompozice
 - Daří se? → můžeme dočasně zrychlit
 - Plán není posloupnost kroků, ale posloupnost ověření

→ MCP protokol

Tvorba agentního flow

Agentické divadlo

- Hledání chyb v kódu
- 3 koncové role:
 - Hledač
 - Oponent
 - Soudce
- 1 orchestrační role
- Zkuste prompt pro tvorbu agentů [/create-agent](#)

Agent Hledač

- Vytvořte agenta Hledače
- Jeho cílem je nalézt co nejvíce chyb
 - Netřeba hledět na falešná hlášení
- Otázky na zamyšlení:
 - Jaký rozsah práce agentovi zadáte?
 - Jaká zjištění má agent reportovat?
 - Jaký model použijete?
 - Jaké tooly dáte k dispozici?
- Otestuje fungování

Jaké jsou vaše
postřehy?

Úprava postupu

- Použijte mé meta agenty na tvorbu agentů
 - <https://github.com/knowitcz/meta-agents> → .github → agents
- Meta Designer – hlavní agent, se kterým diskutovat svůj záměr
 - volá si další meta agenty na konkrétní tvorbu
- Meta Agent Designer – agent na tvorbu agentů
- Meta Skill Designer – agent na tvorbu skillů

Co se má stát?

- Cíl: najít všechny chyby
- Úkol Hledače: nalézt co nejvíc chyb
- Úkol Oponenta: odmítnout všechny nechyby
 - použít logické argumenty
- Úkol Soudce: rozsoudit, zda je dané hlášení skutečně chyba
- Úkol orchestrace: provést správné úkony
 - Hledač → Oponent → Soudce → report

Otázky na zamyšlení

- Jak velký rozsah práce má dělat každý agent?
- Jaké informace si mají předávat?
- Jaké modely použijete?
- Co když si agent není jistý svým rozhodnutím?
- Co mají a **nemají** agenti dělat?
- Jak má vypadat report?
- Je vhodné upravit vytvořeného Hledače?

Hotovo?

Spustíte agenty nad projektem

Diskuze

Jaká jsou zjištění agentů?

Jaké jsou vaše dojmy z divadla?

→ AI v SDLC

Jak publikovat zjištěné
chyby?

Vlastní skill – formát reportování chyb

- Inspirujte se v *docs/bug-reporting/report-helper.md*
 - Vytvořte šablonu, co musí bug report obsahovat
 - Detailně popište kroky vedoucí k publikaci chyb:
 - MCP
 - soubor
 - ...
- Definujte jasný popis, kdy by se měl skill aktivovat
- Vyzkoušejte použít `/create-skill` prompt

→ Validace SKILL.md

Diskuze

Co všechno jste ve skillu upravili?

→ AI Act

Materiály budou ve složce `docs/slides`

KNOW IT

Děkuji za pozornost